

Assignment 15(CSB24084)

```

#include <stdio.h>
#include <pthread.h>

#define NUM_THREADS 4
#define INCREMENTS 100000

long long counter = 0;

pthread_mutex_t lock;

void *increment_without_mutex(void *arg)
{
    for (int i = 0; i < INCREMENTS; i++)
    {
        counter++;
    }

    return NULL;
}

void *increment_with_mutex(void *arg)
{
    for (int i = 0; i < INCREMENTS; i++)
    {
        pthread_mutex_lock(&lock);

        counter++;

        pthread_mutex_unlock(&lock);
    }

    return NULL;
}

int main()
{
    pthread_t threads[NUM_THREADS];

    counter = 0;

    printf("Running WITHOUT mutex synchronization...\n");

    for (int i = 0; i < NUM_THREADS; i++)
    {

```

```

        pthread_create(&threads[i], NULL,
                      increment_without_mutex, NULL);
    }

    for (int i = 0; i < NUM_THREADS; i++)
    {
        pthread_join(threads[i], NULL);
    }

    printf("Final Counter Value (Without Mutex) = %lld\n", counter);
    printf("Expected Value          = %d\n\n",
          NUM_THREADS * INCREMENTS);

    counter = 0;

    printf("Running WITH mutex synchronization...\n");

    pthread_mutex_init(&lock, NULL);

    for (int i = 0; i < NUM_THREADS; i++)
    {
        pthread_create(&threads[i], NULL,
                      increment_with_mutex, NULL);
    }

    for (int i = 0; i < NUM_THREADS; i++)
    {
        pthread_join(threads[i], NULL);
    }

    pthread_mutex_destroy(&lock);

    printf("Final Counter Value (With Mutex) = %lld\n", counter);
    printf("Expected Value          = %d\n",
          NUM_THREADS * INCREMENTS);

    return 0;
}

```

A race condition occurs when multiple threads access and modify the shared counter variable simultaneously without synchronization. The statement `counter++` is not executed as a single operation; it involves reading, incrementing, and writing the value back. When several threads perform these steps at the same time, some updates may be lost, resulting in an incorrect final

counter value. A mutex solves this problem by allowing only one thread to access the critical section at a time using `pthread_mutex_lock()` and `pthread_mutex_unlock()`. This ensures safe access to the shared variable and produces the correct final counter value.